

Appearance Manifold 플러그인 가이드라인

1. 개요 (Introduction)

Appearance Manifold는 텍스처의 물리적 속성(BaseColor, Specular, Roughness)을 7차원 매니폴드 공간으로 매핑하여, 시각적으로 가장 자연스러운 풍화 경로(Trajectory)를 생성하고 런타임에 실시간으로 보간(Interpolation)하는 시뮬레이션 시스템입니다.

- 주요 특징: 가우시안 커널 기반 인접점 연결, Dijkstra 알고리즘을 통한 최지선 거리 측정, MDS 기반 차원 축소 및 비동기 파이썬 연산 파이프라인.

2. 사용자 가이드 (User Guide)

"어떻게 풍화 효과를 설정하고 적용하는가?"에 집중합니다.

2.1 퀵 스타트 (Quick Start)

- 컴포넌트 부착: 풍화를 적용할 액터에 Weathering Manager Component를 추가합니다.
- 베이스 샘플 할당: 컴포넌트에 풍화 상태가 가장 양호한 기본 텍스처를 프로퍼티에 할당합니다.
- 키프레임 캡처:
 - Clear Weathering Textures 버튼을 클릭하여 내부 데이터를 초기화하고 기본 텍스처 정보를 풍화 로직에 포함합니다.
 - Gamma-ton 시뮬레이션을 사용하여 기본 상태의 텍스처를 생성합니다(권장 사이즈: 512x512).
 - Get New Weathered Texture 버튼을 클릭하여 현재 상태를 키프레임으로 저장합니다.
 - 팁: 풍화 과정을 부드럽게 만들기 위해 3회 미만의 키프레임 조합을 권장합니다.
- 경로 생성: Generate Trajectory 버튼을 클릭합니다. (내부적으로 파이썬 파이프라인이 실행되어 풍화 경로를 계산합니다.)
- 시뮬레이션: 게임 실행 시, 컴포넌트의 풍화 속도(Speed)와 스텝(Step) 파라미터를 조절하여 실시간 풍화 효과를 확인합니다.

3. 개발자 가이드 (Technical Guide)

"파이프라인이 어떻게 동작하고 확장하는가?"에 집중합니다.

3.1 시스템 아키텍처

본 플러그인은 C++의 ParallelFor와 Python(NumPy/SciPy) 간의 비동기 바이너리 데이터 교환 방식을 채택하고 있습니다.

3.2 핵심 데이터 관리 경로

플러그인은 프로젝트의 Saved 폴더 내에 데이터를 관리하며, 다음 경로에서 결과물을 직접 확인할 수 있습니다.

- **Saved/WeatheringResults/**: 풍화 궤적 연산을 위한 키프레임 PNG 및 결과 데이터 저장.
- **Saved/TensorCache** : 런타임 보관에 사용되는 7차원 텐서 데이터(.bin) 저장.

3.3 확장 및 커스터마이징

- 파이썬 연산 모듈: `MainManager.py`를 통해 연산 로직을 확장할 수 있습니다.
 - `export_to_unreal_binary` 함수를 통해 데이터를 직렬화합니다.
 - 헤더 포맷: `[T(int32)][N(int32)]` (T: 노드 수, N: 샘플 수) 구조를 유지해야 C++ 모듈에서 정상적으로 읽을 수 있습니다.
- C++ 유틸리티: `AppearanceManifoldBlueprintFunctionLibrary`를 통해 텍스처 데이터 변환 및 바이너리 로드 기능을 확장 가능합니다.

4. 트러블슈팅 (Troubleshooting)

- **Q:** 파이썬 모듈이 설치되지 않아요.
 - **A:** 플러그인 로드 시 `StartupModule()`에서 `EnsurePythonDependenciesInstalled()`가 호출됩니다. 인터넷 연결 상태를 확인하고 Output Log에서 설치 과정을 모니터링하세요.
 - **A:** 만약 어떤 동작도 먹히지 않는다면, 언리얼 파이썬 내부에 `numpy`, `scipy`, `scikit-learn` 모듈을 직접 설치하시는 방법을 권장합니다.
- **Q:** 런타임 보관 결과가 이상합니다.
 - **A:** 키프레임 데이터 간의 해상도가 일치하는지 확인하세요. 모든 소스 텍스처는 **512x512** 사이즈를 기준으로 연산될 때 최적의 성능을 보입니다.
 - **A:** 특히 감마톤 시뮬레이션 결과와 베이스 샘플 텍스처의 사이즈가 맞지 않는 경우 결과가 제대로 나오지 않습니다.

[Technical Whitepaper] Appearance Manifold: 런타임 풍화 연산 엔진

본 시스템은 정적 텍스처 데이터셋을 7차원 특징 공간(**7D Feature Space**) 상의 매니폴드로 간주하고, 이를 비선형 차원 축소와 그래프 이론을 통해 재구성하여 실시간 보관을 수행합니다.

1. Manifold 그래프 구성 및 최적화 (Graph Construction)

시스템의 시작점은 **KNN.py**를 통한 매니폴드 그래프 구축입니다.

- **Adaptive KNN:** 7차원(Base RGB, Spec RGB, Roughness) 샘플 데이터에 대해 K-최근접 이웃을 탐색합니다. 이때 **epsilon** 규칙을 적용하여 노이즈 엣지를 제거하고, 풍화 점수(Weathering Score)의 차분을 가중치(**w**)에 반영합니다.

- $w_{\{ij\}} = d_{\{ij\}} \cdot \phi(\Delta s_{\{ij\}})$ (ϕ : 풍화 방향에 따른 가중치 보정 함수)
- 이를 통해 단순 유클리드 거리가 아닌, 풍화의 물리적 전이 방향성을 포함한 매니폴드 그래프를 확보합니다.

2. 측지선 거리 측정 및 저차원 매핑 (Geodesic & MDS)

곡면 위에서의 실제 이동 거리를 산출하기 위해 `geodesic_backend.cpp`와 `MDS.py`가 협업합니다.

- **Dijkstra** 기반 전역 측지선 산출: 그래프 내의 모든 노드 쌍에 대해 최단 경로(측지선)를 계산합니다. C++의 `ParallelFor`와 `Eigen` 라이브러리를 통해 이 고비용 연산을 병렬 처리합니다.
- **MDS(Multidimensional Scaling)**: 산출된 거리 행렬 D 를 고전적 MDS 기법(Double Centering)을 통해 3차원 공간으로 투영합니다. 이는 데이터의 위상적 구조를 최대한 보존하면서 풍화 경로 계산을 용이하게 하는 저차원 임베딩 과정입니다.
- + `geodesic_backend.cpp`의 경우, `pybind11`을 활용하여 파이썬에서 활용할 수 있는 모듈로 변환합니다. (w/ CMake)

3. 궤적 생성 및 시뮬레이션 (Trajectory Acquisition)

`Trajectory.py`는 최적화된 저차원 공간 내에서 풍화의 시퀀스를 확정합니다.

- 최단 경로 탐색: 시뮬레이션의 시작점(S)과 끝점(E) 사이의 최단 경로를 탐색하여 이를 풍화 궤적(Trajectory)으로 정의합니다.
- 재샘플링(Resampling): 궤적의 이동 거리(L)를 기준으로 균등 간격의 T 개 샘플로 보간하여, 런타임에 일정한 속도로 풍화 효과가 진행되도록 정규화합니다.

4. 하이브리드 리컨스트럭션 (Interpolation Pipeline)

런타임 보간은 `Reconstructor.py`의 핵심 로직을 따릅니다.

- 가우시안 커널 기반 가중 보간: 실시간 풍화 상태에서의 픽셀 값은 단순히 선형 보간되지 않습니다.
 - 현재 타겟 지점과 가장 가까운 샘플 데이터들을 추출하고, 가우시안 가중치
$$e^{-\frac{\{dist\}^2}{2\sigma^2}}$$
를 적용하여 다중 샘플을 혼합합니다.
- **Hybrid Reconstruction**: 가장 가까운 샘플(Nearest)의 물리적 정확성과 주변 샘플의 가중 보간된(Blended) 부드러움을 믹싱하여, 풍화 단계에서의 시각적 왜곡(Artifact)을 최소화합니다.

통합 워크플로우: MainManager와의 상호작용

`MainManager.py`는 위 연산들을 비동기 오케스트레이터(Orchestrator)로서 통제합니다.

1. 데이터 동기화: 언리얼에서 텍스처를 PNG로 추출하고, 이를 `Utility.py`로 7차원 텐서화합니다.
2. 연산 루프: `MainManager` 내의 `WeatheringPipeline`은 언리얼의 `Slate Pre-Tick` 콜백을 이용해 엔진 메인 스레드를 차단하지 않고, 백그라운드 파이썬 연산의 완료 여부를 폴링합니다.

3. 바이너리 배포: 최종적으로 확보된 궤적 데이터는 `export_to_unreal_binary`를 통해 `[T][N]` 헤더를 포함한 바이너리 스트림으로 변환됩니다. 이는 언리얼의 `FBufferArchive`와 호환되어 런타임에 메모리로 즉시 로드됩니다.